

Doc. no. P0085R1

Audience: EWG, LEWG

Date: 12-05-2025

Project: ISO JTC1/SC22/WG21: Programming Language C++, evolution group

Reply to: Jolly Chen <Jolly.Chen@cern.ch>, Axel Naumann <Axel.Naumann@cern.ch>, Michael Jonker <Michael.Jonker@cern.ch>,

Oo... adding a coherent character sequence to begin octal-literals

Revision History

Since [P0085R0](#)

- Rebased Technical Specification onto [N5008](#)
- Align grammar with C's [N3353](#)
- Propose deprecation of original octal literal
- Removed proposed change "Technical Specification C run time library functions: strol, scanf". This has been handled by C in [N3353](#).
- Removed proposed change to Addendum, character literals. Octals in escape sequences have been added by [P2290R3](#)
- Added open question on `std::format` with the `#` option for octal literals

Proposal

Proposal to add `0o` and `0O` as an alternative (and preferred) sequence to introduce octal-literals and deprecate the old prefix `0`.

The syntax rule to interpret integer literals starting with a zero as octal-literals might be called a "historical mistake". It can be easily misunderstood by novice programmers and can lead to surprising errors.

To allow future generations (of developers if not compilers) to correct this feature, we propose to add the character sequence `0o` and `0O` as preferred sequences to introduce an octal-literal. The prefix `0o` follows the model set by the prefix `0x` to introduce a hex-literal, and (since C++14) `0b` to introduce a binary-literal. Compilers will be able to warn about the use of the deprecated `0` prefix, and eventually (on the scale of a few releases) the old syntax can be removed.

From <http://en.wikipedia.org/wiki/Octal>: "Newer languages have been abandoning the prefix `0`, as decimal numbers are often represented with leading zeroes. The prefix `q` was introduced to avoid the prefix `o` being mistaken for a zero, while the prefix `0o` was introduced to avoid starting a numerical literal with an alphabetic character (like `o` or `q`), since these might cause the literal to be confused with a variable name. The prefix `0o` also follows the model set by the prefix `0x` used for hexadecimal literals in the C language; it is supported by Haskell,[11] OCaml,[12] Perl 6,[13] Python as of version 3.0,[14] Ruby,[15] Tcl as of version 9,[16] and it is intended to be supported by ECMAScript 6[17] (the prefix `0` has been discouraged in ECMAScript 3 and dropped in ECMAScript 5[18])."

Examples

The following literals all specify the same number.

Literal	Before	After
Hex	0x2A	0x2A
Binary	0b00101010	0b00101010
Octal	052	0o52

The old octal literal 052 will continue to remain valid but deprecated.

Effects on existing code

This proposal deprecates the existing error-prone syntax rule for integer literals starting with a zero, following the example of C ([N3353](#)). It introduces 0o as a new prefix for octal-literals. Additionally, it deprecates the use of non-zero octal integer literals without the prefix 0o or 0O, similar to C declaring this an obsolescent feature.

Under the current standard, any sequence starting with 0o is illegal. Consequently, the proposed additions 0o and 0O will not break existing code.

Technical Specification

To match [N3353](#), make the following edits (relative to [N5008](#)) with **insertions** and **removals** marked like so:

5.13.2 Integer literals [lex.icon]

```
octal-literal
0
octal-literal â€™opt octal-digit
prefixed-octal-literal
unprefixed-octal-literal

unprefixed-octal-literal:
0
0 â€™opt octal-digit-sequence

prefixed-octal-literal:
octal-prefix octal-digit-sequence
```

Before *hexadecimal-prefix* insert

```
octal-prefix: one of
0o 0O
```

Before *hexadecimal-digit-sequence* insert

```
octal-digit-sequence:
octal-digit
octal-digit-sequence â€™opt octal-digit
```

2 [Example 1 : The number twelve can be written 12, ~~014~~, 0o14, 0XC, or 0b1100. The integer-literals 1048576, 1â€™_{opt}048â€™_{opt}576, 0X100000, 0x10â€™_{opt}0000, and ~~0â€™_{opt}004â€™_{opt}000â€™_{opt}000~~ 0o0â€™_{opt}004â€™_{opt}000â€™_{opt}000, all have the same value. â€™” end example]

31.12.8.4 Enum class perm [fs.enum.perms]

Table 149 “Enum class perms [tab:fs.enum.perms]

Name	Value (octal)	POSIX macro	Definition or notes
none	00		There are no permissions set for the file.
owner_read	00400	S_IRUSR	Read permission, owner
owner_write	00200	S_IWUSR	Write permission, owner
owner_exec	00100	S_IXUSR	Execute/search permission, owner
owner_all	00700	S_IRWXU	Read, write, execute/search by owner; owner_read owner_write owner_exec
group_read	0040	S_IRGRP	Read permission, group
group_write	0020	S_IWGRP	Write permission, group
group_exec	0010	S_IXGRP	Execute/search permission, group
group_all	0070	S_IRWXG	Read, write, execute/search by group; group_read group_write group_exec
others_read	004	S_IROTH	Read permission, others
others_write	002	S_IWOTH	Write permission, others
others_exec	001	S_IXOTH	Execute/search permission, others
others_all	007	S_IRWXO	Read, write, execute/search by others; others_read others_write others_exec
all	00777		owner_all group_all others_all
set_uid	004000	S_ISUID	Set-user-ID on execution
set_gid	002000	S_ISGID	Set-group-ID on execution
sticky_bit	001000	S_ISVTX	Operating system dependent.
mask	007777		all set_uid set_gid sticky_bit
unknown	0xFFFF		The permissions are not known, such as when a file_status object is created without specifying the permissions

A.3 Lexical conventions [gram.lex]

octal-literal

~~0~~
~~octal-literal~~ $\hat{=}$ ~~opt~~ ~~octal-digit~~

prefixed-octal-literal
unprefixed-octal-literal

unprefixed-octal-literal:

0
0 $\hat{=}$ opt octal-digit

prefixed-octal-literal:
octal-prefix octal-digit-sequence

Before *hexadecimal-prefix* insert

octal-prefix: one of
00

00

Before *hexadecimal-digit-sequence* insert

octal-digit-sequence:
octal-digit
octal-digit-sequence $\hat{\epsilon}_{opt}$ *octal-digit*

Annex D (normative) Compatibility features [depr]

Add a new section

D??. **Deprecated unprefix octal literal [depr.oct]**

1 An octal literal ([lex.icon],[gram.lex]) of the form

unprefixed-octal-literal

is deprecated.

Open Question: `std::format`

In 28.5.2.2 Standard format specifiers [format.string.std], we have the following example:

21 *The available integer presentation types for integral types other than bool and charT are specified in Table 102. [Example 4 :*

```
string s0 = format("{} ", 42); // value of s0 is "42"
string s1 = format("{0:b} {0:d} {0:o} {0:x}", 42); // value of s1 is "101010 42 52 2a"
string s2 = format("{0:#x} {0:#X}", 42); // value of s2 is "0x2a 0X2A"
string s3 = format("{:L}", 1234); // value of s3 can be "1,234"
// (depending on the locale)
```

end example]

The example shows that `std::format` returns a consistent base prefix for the alternate form `#` option with a hexadecimal type -- `0x` for `#x` and `0X` for `#X`. However, the same is not true for the `#` option with an octal type, where we would have:

```
std::string s1 = std::format("{:#o}", 042); // value of s1 is "042"
std::string s2 = std::format("{:#o}", 0o42); // value of s2 is "042"
// The option #0 does not exist
```

To follow the deprecation of the *unprefixed-octal-literal*, we would prefer a behavior change in the output of `std::format` with `#o` to use the prefix `0o`. By using the search term "`std::format #o` language:C++" on GitHub, we found that the format specifier `#o` was used in only 23 files and out of those, 13 are educational examples and 5 are test suites. This could indicate that a behavior change will have minimal impact on existing code. Given the improved clarity of the new prefix (with potential security implications), we recommend this backward-incompatible change.

[N3353](#) calls out this problem, but does not attempt to solve it. As backward-compatible alternatives, we propose several options:

Option 1

Add the #0 option, to return 0o

- Con: inconsistent with the convention that e.g., #X returns upper-case 0X
- Con: can be confused with the option #0

Option 2

Add the #0 option, to return 00

- Pro: consistent with the convention that e.g., #X returns upper-case 0X
- Con: 00 is less readable than 0o, so we do not want to encourage its use because 00 can be confused with the letter O

Option 3

Add the ##o and ##0 options, to return 0o and 00 respectively

- Rationale: #o asks for a single character prefix and ##o asks for a 2 character prefix
- Pro: can format for both 0o and 00
- Con: maybe a bit obscure

Option 4

Do Nothing

- Con: std::format only outputs the deprecated octal-literal

Acknowledgements

Thanks to Erich Keane for reviewing the draft. The document style was borrowed from Doc. no. [N4340](#)